Syntax & Semantics

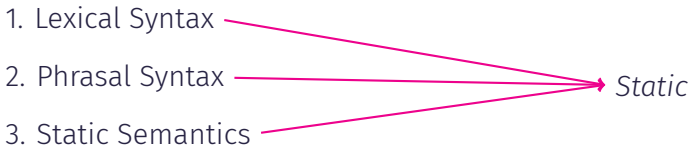
Jibesh Patra, Soumyajit Dey

Week 5 – Software Engineering – CS20202 – Spring 2026 – IIT Kharagpur

Introduction

Processing of Programming Languages

1. Lexical Syntax
2. Phrasal Syntax
3. Static Semantics
4. Dynamic Semantics



Errors on Processing

- Static errors: Errors occurring during the static layers.
  - lexer
  - parser
  - type checker
- Dynamic error: Errors during program execution.

Use OCaml esque languages to explain the features.

Lexical Syntax

Lexical Syntax

The fixed set of *characters* a programming language can be written with.

- White space characters:
  - space
  - carriage return
  - horizontal tab
- Fixed classes of words:
  - Keywords
  - Operators
  - Literals
  - Identifiers

```
1 let ü x = x + 2
2 let res = ü 23
```

Keywords

In OCaml, the following are some of the keywords:

```
let rec in if then else function
( ) = : → * ,
```

Operators

In OCaml, the following are some of the operators:

`+ - / mod <> < ⩽ > ⩾`

Literals, Identifiers & Comments

- Booleans: `true, false`
- Integers: `0, 1, 293`
- String: `"Hello", "Foo"`
- Identifiers: starts with lowercase letters, can contain digits and `_`.
- Comments: `(* ... *)` count as white space.
- `*` & `=` serve both as *keyword* and *operator*.

Phrasal Syntax

Phrasal Syntax

- Language don’t see programs as sequence of characters or words.
- As syntactic objects called phrases.
  - Best understood as syntax trees
- Phrasal syntax organizes the phrases of the language into different syntactic classes (OCaml):
  - types
  - expressions
  - declarations
  - programs
- Every phrase belong to exactly one syntactic class.

Represent Using Mathematical Symbols

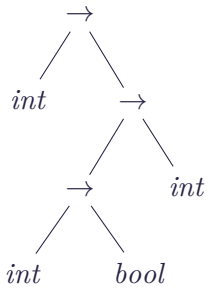
OCaml Symbols	Corresponding Mathematical Symbols
`+ - / mod`	$+ \quad - \quad / \quad \%$
`*`	$\cdot$ or $\times$
`= <>`	$= \quad \neq$
`< <= > >=`	$< \quad \leq \quad > \quad \geq$
$\rightarrow$	$\rightarrow$

Types

- Base types: `int, bool, string`.
- Types can be combined into *function types* and *tuple types* using the symbols $\rightarrow$ and $\times$.

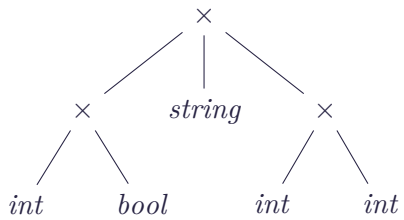
$$int \rightarrow (int \rightarrow bool) \rightarrow int$$

$$int \rightarrow ((int \rightarrow bool) \rightarrow int)$$

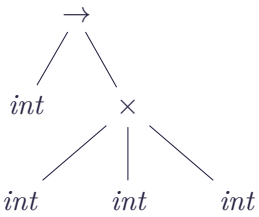


Types (Continued)

$(int \times bool) \times string \times (int \times int)$



$int \to int \times int \times int$



$int \times int \times int \quad \not\to \quad int \times (int \times int)$

Backus-Naur Form (BNF) Grammar

BNF grammar:

$$
\begin{aligned}
\langle type \rangle &::= \langle base\ type \rangle \\
&\quad \mid \langle type \rangle \rightarrow \langle type \rangle \\
&\quad \mid \langle type \rangle_1 \times \cdots \times \langle type \rangle_n \quad (n \geq 2) \\
\langle base\ type \rangle &::= int \mid bool \mid string
\end{aligned}
$$

function type $\Rightarrow \langle type \rangle \rightarrow \langle type \rangle$

tuple type $\Rightarrow \langle type \rangle_1 \times \cdots \times \langle type \rangle_n \quad (n \geq 2)$

Expressions - Grammar

$$
\begin{aligned}
\langle \textit{expression} \rangle ::= & \\
& \mid \langle \textit{atomic expression} \rangle \\
& \mid \langle \textit{operator application} \rangle \\
& \mid \langle \textit{function application} \rangle \\
& \mid \langle \textit{conditional} \rangle \\
& \mid \langle \textit{tuple expression} \rangle \\
& \mid \langle \textit{lambda expression} \rangle \\
& \mid \langle \textit{let expression} \rangle
\end{aligned}
$$

$$
\langle \textit{atomic expression} \rangle ::= \langle \textit{literal} \rangle \mid \langle \textit{identifier} \rangle
$$

Expressions - Grammar (Continued)

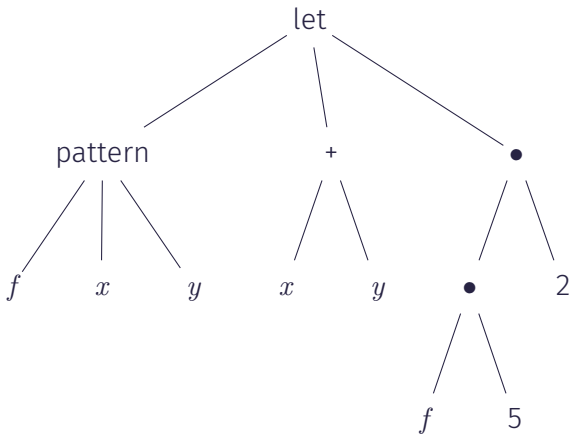
$$
\begin{aligned}
\langle \textit{operator application} \rangle ::= & \\
& \mid \langle \textit{expression} \rangle \langle \textit{operator} \rangle \langle \textit{expression} \rangle \\
& \mid \langle \textit{operator} \rangle \langle \textit{expression} \rangle \\
\langle \textit{operator} \rangle ::= & + \mid - \mid \cdot \mid / \mid \% \mid = \mid \neq \mid < \mid \leq \mid > \mid \geq \\
\langle \textit{function application} \rangle ::= & \langle \textit{expression} \rangle \ \langle \textit{expression} \rangle \\
\langle \textit{conditional} \rangle ::= & \texttt{if} \ \langle \textit{expression} \rangle \ \texttt{then} \ \langle \textit{expression} \rangle \ \texttt{else} \ \langle \textit{expression} \rangle \\
\langle \textit{tuple expression} \rangle ::= & (\langle \textit{expression} \rangle_1 \ , \ \ldots \ , \ \langle \textit{expression} \rangle_n) \quad (n \geq 2) \\
\langle \textit{lambda expression} \rangle ::= & \texttt{fun} \ \langle \textit{pattern} \rangle_1 \ \ldots \ \langle \textit{pattern} \rangle_n \rightarrow \langle \textit{expression} \rangle \\
\langle \textit{let expression} \rangle ::= & \texttt{let} \ \langle \textit{let pattern} \rangle = \langle \textit{expression} \rangle \ \texttt{in} \ \langle \textit{expression} \rangle
\end{aligned}
$$

Expressions - Grammar (Continued)

$$
\begin{aligned}
\langle \textit{let pattern} \rangle ::= & \\
& \mid \langle \textit{pattern} \rangle \\
& \mid \langle \textit{identifier} \rangle\ \langle \textit{pattern} \rangle_1\ \ldots\ \langle \textit{pattern} \rangle_n \quad (n \geq 1) \\
& \mid \texttt{rec}\ \langle \textit{identifier} \rangle\ \langle \textit{pattern} \rangle_1\ \ldots\ \langle \textit{pattern} \rangle_n \quad (n \geq 0) \\
\langle \textit{pattern} \rangle ::= & \\
& \mid \langle \textit{identifier} \rangle \\
& \mid (\langle \textit{pattern} \rangle_1\ ,\ \ldots\ ,\ \langle \textit{pattern} \rangle_n) \quad (n \geq 2)
\end{aligned}
$$

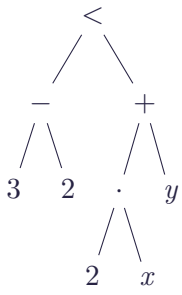
Expressions Syntax Tree

```
1 let f x y = x + y in f 5 2
```



May Omit Parentheses in Lexical Representation

$$(3 - 2) < ((2 \cdot x) + y) \quad \rightsquigarrow \quad 3 - 2 < 2 \cdot x + y$$



Parentheses for Binary Operators

Levels to distinguish

$$= \quad \neq \quad < \quad \leq \quad > \quad \geq$$

$$+ \quad -$$

$$\cdot \quad / \quad \%$$

$$3 - 2 < 2 \cdot x + y \quad \rightsquigarrow \quad (3 - 2) < ((2 \cdot x) + y)$$

$$e_1 + e_2 + e_3 - e_4 - e_5 \quad \rightsquigarrow \quad (((e_1 + e_2) + e_3) - e_4) - e_5$$

$$1 + + + 5 - - + -7 \quad \rightsquigarrow \quad (1 + (+(+5))) - (-(+(-7)))$$

Declarations and Programs

- A program is a sequence of declarations.

$$\langle program \rangle ::= \langle declaration \rangle_1 \ \ldots \ \langle declaration \rangle_n \quad (n \geq 0)$$
$$\langle declaration \rangle ::= \text{let } \langle let\ pattern \rangle = \langle expression \rangle$$

- The previously seen let expressions can be modelled as

$$\langle let\ expression \rangle ::= \langle declaration \rangle \text{ in } \langle expression \rangle$$

Derivation Systems

Derivation Systems

- The semantic layers of programming language are described with derivation systems.
- Derivation Systems $\Rightarrow$ Systems of inference rules.
- Assume x, y are integers for deriving $x \dot{<} z$ with the following two inference rules:

$$\frac{}{x \dot{<} x+1}$$

$$\frac{x \dot{<} y \quad y \dot{<} z}{x \dot{<} z}$$

$$\frac{\dfrac{3 \dot{<} 4 \qquad \dfrac{4 \dot{<} 5 \qquad 5 \dot{<} 6}{4 \dot{<} 6}}{3 \dot{<} 6}$$

$$\frac{\dfrac{\dfrac{}{3 \dot{<} 4} \quad \dfrac{}{4 \dot{<} 5}}{3 \dot{<} 5} \qquad \dfrac{}{5 \dot{<} 6}}{3 \dot{<} 6}$$

Inference Rule

General form of inference rules where **premises** are $P_1, \ldots, P_n$ and C is the **conclusion**.

$$\frac{P_1 \quad \ldots \quad P_n}{C}$$

Language

- Types

$$t ::= int \mid bool \mid string \mid t_1 \to t_2 \mid t_1 \times \cdots \times t_n \quad n \geq 2$$

- Grammar

$$\begin{aligned}
e ::= {} & c \mid x \mid e_1 \; o \; e_2 \mid e_1 \; e_2 \\
& \mid \texttt{IF} \;\; e_1 \;\; \texttt{THEN} \;\; e_2 \;\; \texttt{ELSE} \;\; e_3 \\
& \mid (e_1, \ldots, e_n) && n \geq 2 \\
& \mid \pi_n \, e && n \geq 1 \\
& \mid \texttt{LET} \;\; x = e_1 \;\; \texttt{IN} \;\; e_2 \\
& \mid \lambda x.e \\
& \mid \texttt{LET REC} \;\; f \;\; x = e_1 \;\; \texttt{IN} \;\; e_2
\end{aligned}$$

Grammar Details

- c ranges over **constants** (literals). The values are of types *int, bool, string*.
- x and f ranges over **variables** (identifiers).
- o ranges over **operators**. Here the only operator type is *binary operator*.
- Variables introduced by lambda expression & let expressions are **local variables**.
- Tuples *patterns* are represented by projections π_n, yielding nth component of a tuple.

Static Semantics

Typing Judgment

$$E \vdash e : t$$

- The above typing judgement says that in environment E, the expression e has type t.
- An **environment** is a collection of bindings $x : t$ mapping variables to types.
- We assume that in each environment, every varible at most one binding.
- **Example** $x : int, y : bool \vdash \texttt{IF } y \texttt{ THEN } x \texttt{ ELSE } 0 : int$
- Lambda expressions and let expressions *update* the existing environment E with a binding $x : t$. If E does not contain a binding for x, the binding $x : t$ is added else the existing binding is replaced.

Tacit Premises

- A condition or assumption that is not written explicitly in the typing rule, but is implicitly assumed to hold for the judgment to make sense.
- **Constants:** A constant c has type t according to the language definitions. Examples $\quad true : bool$ & $3 : int$.
- **Variables:** All free variables of an expression are in the environment $(x : t) \in E$ i.e., the environment E contains a binding for the variable x.
- **Operator Applications:** An operator $o : t_1 \rightarrow t_2 \rightarrow t$ is defined to take arguments of types t_1 & t_2 for operator o. Examples $\quad + : int \rightarrow int \rightarrow int$ & $< : int \rightarrow int \rightarrow bool$

Typing Judgment (Continued)

$$x : int, y : bool \vdash \texttt{IF}\ \ y\ \ \texttt{THEN}\ \ x\ \ \texttt{ELSE}\ \ 0 : int$$

$$\frac{\overline{E \vdash y : bool} \quad \overline{E \vdash x : int} \quad \overline{E \vdash 0 : int}}{E \vdash \texttt{IF}\ \ y\ \ \texttt{THEN}\ \ x\ \ \texttt{ELSE}\ \ 0 : int}$$

Typing Rules

$$\frac{c : t}{E \vdash c : t}$$

$$\frac{(x : t) \in E}{E \vdash x : t}$$

$$\frac{o : t_1 \to t_2 \to t \qquad E \vdash e_1 : t_1 \qquad E \vdash e_2 : t_2}{E \vdash e_1 \; o \; e_2 : t}$$

$$\frac{E \vdash e_1 : t_2 \to t \qquad E \vdash e_2 : t_2}{E \vdash e_1 \; e_2 : t}$$

$$\frac{E \vdash e_1 : bool \qquad E \vdash e_2 : t \qquad E \vdash e_3 : t}{E \vdash \texttt{IF} \;\; e_1 \;\; \texttt{THEN} \;\; e_2 \;\; \texttt{ELSE} \;\; e_3 : t}$$

Typing Rules (Continued)

$$\frac{E \vdash e_1 : t_1 \quad \ldots \quad E \vdash e_n : t_n}{E \vdash (e_1, \ldots, e_n) : t_1 \times \cdots \times t_n}$$

$$\frac{E \vdash e_1 : t_1 \qquad E, x : t_1 \vdash e_2 : t}{E \vdash \texttt{LET}\ \ x = e_1\ \ \texttt{IN}\ \ e_2 : t}$$

$$\frac{E \vdash e : t_1 \times \cdots \times t_n \qquad 1 \leq i \leq n}{E \vdash \pi_i\, e : t_i}$$

$$\frac{E, x : t_1 \vdash e : t_2}{E \vdash \lambda x.e : t_1 \to t_2}$$

$$\frac{E, f : t_1 \to t_2, x : t_1 \vdash e_1 : t_2 \qquad E, f : t_1 \to t_2 \vdash e_2 : t}{E \vdash \texttt{LET REC}\ \ f x = e_1\ \ \texttt{IN}\ \ e_2 : t}$$

Well-Typedness

- Given an environment E and an expression e, if the judgement $E \vdash e : t$ is derivable for some type t we say the expression e is **well typed**.
- A expression e is **ill-typed** in the environment E if there is no type t such that the judgement $E \vdash e : t$ is derivable.
- OCaml only accepts well-typed programs.
- A system of typing rules is referred to as **type system**.

Types in Lambda Expressions and Let Expressions

- Lambda expressions and recursive let expressions *guess* the types for the local variables they introduce.

$$\lambda x.e \quad \rightsquigarrow \quad \lambda x : t.e$$

$$\texttt{LET REC } f x = e_1 \texttt{ IN } e_2 \quad \rightsquigarrow \quad \texttt{LET REC } f(x : t_1) : t_2 = e_1 \texttt{ IN } e_2$$

- For non-recursive let expressions, no *guessing* is needed.

$$\frac{E \vdash e_1 : t_1 \qquad E, x : t_1 \vdash e_2 : t}{E \vdash \texttt{LET } x = e_1 \texttt{ IN } e_2 : t}$$

Type Derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{[] \vdash 1 : \text{int}} \quad \dfrac{}{[] \vdash \text{true} : \text{bool}}}{[] \vdash (1, \text{true}) : \text{int} \times \text{bool}} \quad \dfrac{}{[] \vdash 3 : \text{int}}}{[] \vdash ((1, \text{true}), 3) : (\text{int} \times \text{bool}) \times \text{int}}}{[] \vdash \pi_1((1, \text{true}), 3) : \text{int} \times \text{bool}}$$

$$\dfrac{\dfrac{\dfrac{}{x : t_2 \vdash x : t_2}}{x : t_1 \vdash \lambda x.x : t_2 \to t_2}}{[] \vdash \lambda x.\lambda x.x : t_1 \to t_2 \to t_2}$$